

ReverseBids MVP — Code Review, Bug Fixes & Verification

Reverse Auction Codebase (React + Firebase)

Prepared for Goodrich Ingredients & Chemicals / ReverseBids.net

July 2, 2026

1. Summary

The original hand-off (a DeepSeek-generated MVP scaffold) had a working happy path but several bugs that would have caused **real financial and security problems** in production, plus one file that was missing entirely. This report documents each issue found, the fix applied, and a regression test run confirming the fixed logic behaves correctly across boundary cases.

The ABO Stress Platform section of the original document was wireframes only (no code), so there is nothing to fix there — it is reproduced as-is at the end of this report for reference.

Severity breakdown

#	Issue	Severity	File(s)
1	Client could write bids directly to Firestore, bypassing all business-logic checks	Critical (security)	firestore.rules, BidForm.jsx
2	notifications.js was referenced by index.js but never actually included — deploy would crash	Critical (deploy-breaking)	functions/src/notifications.js
3	No winner was ever determined when an auction closed	High (functional)	closeAuction.js
4	Bid history exposed real supplier names to competitors, defeating the blind-bidding requirement	High (business logic)	BidHistory.jsx, validateBid.js
5	Floating-point money comparisons could accept/reject boundary bids incorrectly	Medium (financial correctness)	validateBid.js
6	No anti-sniping protection — a bid in the last second wins with no chance to respond	Medium (functional)	validateBid.js
7	Any authenticated non-buyer was treated as a valid bidder, regardless of their actual account role	Medium (authorization)	AuctionDetail.jsx, validateBid.js
8	NaN/empty bid amounts could slip past client-side validation	Low	BidForm.jsx
9	closeExpiredAuctionsLow (scale) batched unboundedly — silently fails past ~500 due auctions		closeAuction.js

#	Issue	Severity	File(s)
10	User profile documents were broadly readable, and role was writable after account creation	Medium (security)	firestore.rules

2. Detailed Findings & Fixes

2.1 Critical — Client-side bid writes bypassed all validation

The original BidForm.jsx ran a Firestore runTransaction() **directly from the browser**, and the original firestore.rules allowed any authenticated supplier to write currentPrice, bidCount, and bids on an active auction. The validateBid Cloud Function existed in the codebase but the client never called it.

Net effect: every check in BidForm.jsx — minimum decrement, “buyer can’t bid on their own auction”, deadline — was cosmetic. A user with a modified client, or simply curl plus a valid Firebase ID token, could write an arbitrary currentPrice, including a negative number or a price above what should have been allowed, straight into Firestore.

Fix: BidForm.jsx now calls the validateBid callable Cloud Function exclusively. The Firestore rule for auctions now only lets the buyer edit non-economic fields (description, specifications) while active; currentPrice / bidCount / bids can only be written by the Admin SDK inside the Cloud Function, which rules cannot be bypassed by any client.

2.2 Critical — Missing file breaks the entire functions deployment

functions/src/index.js does:

```
const { sendBidNotification, sendAuctionClosedNotification } = require('./notifications');
```

The original hand-off never included notifications.js (the transcript was truncated before that file appeared). In Node/Firebase Functions, a missing require() target throws at module load time, which takes down **every** exported function in that index.js — not just the notification ones. Deploying as-is would have meant validateBid and closeExpiredAuctions never boot either.

Fix: notifications.js is now implemented — sendBidNotification fires on bid-count increase, sendAuctionClosedNotification fires on the active → closed transition and notifies both the buyer and (if one exists) the winning supplier looked up from the new private/winner subcollection.

2.3 High — No winner was ever recorded

closeExpiredAuctions only flipped status to closed. In a “lowest qualified bid wins” model, currentPrice already holds the winning price by construction, but nothing captured *who* that was — the buyer had no way to know which supplier to award the order to without manually reading the raw bid array.

Fix: on close, the function now writes the last (i.e. lowest) bid’s supplier into a restricted auctions/{id}/private/winner document.

2.4 High — Bid history broke blind bidding

BidHistory.jsx rendered bid.supplierName directly, and validateBid.js stored the real display name on every bid. That means every supplier bidding on an auction could see every other supplier’s real identity and their exact bid — which directly contradicts the blind-seller-alias requirement in the ReverseBids product design.

Fix: bids now store a per-auction supplierAlias (“Supplier A”, “Supplier B”, ...) assigned in first-bid order and persisted in auction.supplierAliases. Real identity is only ever written, at close time, into the restricted private/winner doc — readable solely by the buyer and the winning supplier (Firestore rule in §2.10).

2.5 Medium — Floating-point bid comparisons

```
if (amount >= auction.currentPrice) { ... }           // original
const maxBid = auction.currentPrice - auction.minimumBidDecrement;
if (amount > maxBid) { ... }
```

JavaScript floats are binary, not decimal — $100.10 - 0.10$ evaluates to 100.000000000000001 , not 100. Right at a decrement boundary, a bid that should be legal could be rejected (or vice versa, depending on which way the rounding error falls).

Fix: all comparisons now happen in integer paise ($\text{Math.round}(\text{rupees} * 100)$), which has no such rounding error. Verified in the test suite (§3, “boundary bid” case).

2.6 Medium — No anti-sniping protection

The deadline was a fixed timestamp with no response to late activity, so a supplier could submit a winning bid in the last second of the auction, leaving no time for competitors to react — a real defect flagged during earlier validation of this codebase.

Fix: `validateBid.js` now extends the deadline by 2 minutes whenever a valid bid lands within 2 minutes of the current deadline, capped at 10 extensions per auction so it can’t be extended indefinitely.

2.7 Medium — Role was inferred, not verified

```
const isSupplier = !isBuyer && user?.uid; // original AuctionDetail.jsx
```

Any authenticated user who simply wasn’t *this auction’s* buyer was treated as a valid bidder — including someone whose own account role is “buyer.” Nothing checked their actual profile.

Fix: `AuctionDetail.jsx` now reads the user’s real role from their `users/{uid}` document, and `validateBid.js` independently enforces `role === 'supplier'` server-side (the authoritative check; the client check is UX only).

2.8 Low — Invalid bid amounts

`parseFloat('')` is NaN, and `NaN >= x` is always false in JavaScript — so an empty or garbage bid amount slipped past the original guard clauses instead of being rejected.

Fix: explicit `Number.isNaN` / empty-string check added before any comparison.

2.9 Low — Unbounded batch on close

A single Firestore batch write is capped at 500 operations. The original `closeExpiredAuctions` put every due auction into one batch with no chunking — past ~500 simultaneously-expiring auctions, the whole batch.`commit()` throws and **nothing** closes that run, silently, until the next 5-minute tick picks it up (or fails again if the backlog keeps growing).

Fix: due auctions are now processed in chunks of 400.

2.10 Medium — User profile & role security

The original rules let any authenticated buyer or supplier read *any* user’s profile document (name/email), and let a user overwrite their own `role` field on every update — meaning a supplier account could silently promote itself to “buyer.” Fixed: profile reads are now restricted to the

owner, and role is immutable after account creation. A new `auctions/{id}/private/{doc}` rule was added to support the restricted winner-identity document from §2.4.

3. Verification (“Back-Test”)

This is a web application, not a trading strategy, so there’s no historical price series to back-test against. What I did instead was extract the core bid-acceptance logic from the fixed `validate-Bid.js` into a standalone harness and run it against 11 scenarios covering every fix above — equivalent in spirit to a back-test in that it replays representative “history” of bid sequences against the new rules before they go anywhere near production data.

```
$ node test_bid_logic.js
PASS valid bid lowers price and records alias
PASS second supplier gets alias B
PASS bid equal to current price is rejected
PASS bid violating min decrement is rejected
PASS bid exactly at max decrement boundary is accepted (float-precision case)
PASS buyer cannot bid on own auction
PASS buyer-role user cannot bid even on someone else's auction
PASS bid after deadline is rejected
PASS last-minute bid extends deadline (anti-sniping)
PASS extension count is capped
PASS NaN / empty amount rejected
```

All 11 scenarios pass. The full harness is reproduced in §5.8 so it can be re-run (`node test_bid_logic.js`, no dependencies required) whenever the logic changes.

Every rewritten `.jsx/.js` file was also syntax-checked with `esbuild` (zero errors) before being included in this document — see the file-by-file listing in Section 5.

4. Recommended Next Steps (not yet implemented)

These are worth doing but were out of scope for a same-day fix pass:

- **Currency storage:** consider storing prices as integer paise in Firestore directly (rather than converting at comparison time) to remove float risk end-to-end, including in the UI layer.
- **Cloud Function auth for closeExpiredAuctions:** currently a scheduled function with full Admin SDK access — fine as-is, but worth a code comment/runbook noting it's the only writer of status.
- **Rate limiting on validateBid:** add App Check + a per-user cooldown to prevent bid-spam from a single compromised account.
- **ReverseBids.sg:** the planned Singapore Firebase project should reuse this same rules/function logic rather than being hand-copied, to avoid the fixes here silently not propagating.

5. Fixed Source Files

5.1 firestore/firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuthenticated() {
      return request.auth != null;
    }

    function userRole() {
      return get(/databases/{database}/documents/users/{request.auth.uid}).data.role;
    }

    function isBuyer() {
      return isAuthenticated() && userRole() == 'buyer';
    }

    function isSupplier() {
      return isAuthenticated() && userRole() == 'supplier';
    }

    match /users/{userId} {
      // BUG FIX: previously any authenticated buyer/supplier could read ANY user doc,
      // leaking emails/names. Users can only read their own profile; public bidding
      // aliases are handled separately and never expose this document.
      allow read: if isAuthenticated() && request.auth.uid == userId;
      allow create: if isAuthenticated() && request.auth.uid == userId;
      // BUG FIX: role was previously writable by the user on every update, so a
      // supplier could self-promote to buyer. Role is now immutable after creation.
      allow update: if isAuthenticated() && request.auth.uid == userId &&
        request.resource.data.role == resource.data.role;
    }

    match /auctions/{auctionId} {
      allow read: if isAuthenticated();

      allow create: if isAuthenticated() && isBuyer() &&
        request.resource.data.buyerId == request.auth.uid &&
        request.resource.data.status == 'active' &&
        request.resource.data.startingPrice > 0 &&
        request.resource.data.minimumBidDecrement > 0 &&
        request.resource.data.currentPrice == request.resource.data.startingPrice &&
        request.resource.data.bidCount == 0 &&
        request.resource.data.bids.size() == 0 &&
        request.resource.data.deadline is string;

      // BUG FIX (critical): the original rule allowed ANY supplier to write
      // currentPrice / bidCount / bids directly from the client as long as the
      // auction was active. That meant the client-side max-bid / decrement /
      // "buyer can't bid on own auction" checks in BidForm.jsx were purely
      // cosmetic – a modified client (or curl request with a valid ID token)
      // could submit an illegal bid, e.g. a negative price or a price above
```

```

    // the current one, and Firestore would accept it.
    //
    // Bids are now written EXCLUSIVELY by the validateBid Cloud Function
    // using the Admin SDK, which bypasses these rules entirely. Direct
    // client writes to the economic fields are denied. The buyer may still
    // edit non-economic metadata (description/specifications) while active,
    // and status transitions are owned by the closeExpiredAuctions function.
    allow update: if isAuthenticated() && isBuyer() &&
        resource.data.buyerId == request.auth.uid &&
        resource.data.status == 'active' &&
        request.resource.data.diff(resource.data).affectedKeys()
            .hasOnly(['description', 'specifications']);

    allow delete: if false;

    // BUSINESS-LOGIC FIX: ReverseBids auctions are meant to use blind seller
    // aliases (per Goodrich/ReverseBids product spec) – suppliers should not
    // see each other's identities in the public bid history. The original
    // code put supplierName directly into the public `bids` array, which
    // defeats that requirement. Real identity of the WINNING supplier is now
    // written by the Cloud Function into this restricted subcollection,
    // readable only by the buyer and the winning supplier themselves.
    match /private/{doc} {
        allow read: if isAuthenticated() &&
            (get(/databases/${database}/documents/auctions/${auctionId}).data.buyerId == request.auth.uid) &&
            resource.data.winnerId == request.auth.uid);
        allow write: if false; // Admin SDK only
    }
}
}
}
}

```

5.2 functions/src/validateBid.js

```

const functions = require('firebase-functions');
const admin = require('firebase-admin');

// BUG FIX: all money is now compared in integer paise (amount * 100, rounded)
// instead of raw JS floats. `0.1 + 0.2 !== 0.3` in floating point, so the old
// `amount >= currentPrice` / `amount > maxBid` comparisons could accept or
// reject a bid incorrectly right at the boundary. Integers have no such issue.
const toPaise = (rupees) => Math.round(rupees * 100);

// ANTI-SNIPING FIX: if a valid bid lands within this window of the deadline,
// push the deadline back by this many milliseconds. This was previously
// identified as a missing feature – suppliers could snipe a win in the final
// second with no chance for others to respond.
const SNIPE_WINDOW_MS = 2 * 60 * 1000; // 2 minutes
const SNIPE_EXTENSION_MS = 2 * 60 * 1000; // extend by 2 minutes
const MAX_EXTENSIONS = 10; // hard cap so an auction can't run forever

exports.validateBid = functions.https.onCall(async (data, context) => {
    if (!context.auth) {
        throw new functions.https.HttpsError('unauthenticated', 'User must be logged in');
    }

```

```

}

const { auctionId, amount } = data;
const userId = context.auth.uid;

if (!auctionId || typeof amount !== 'number' || !isFinite(amount) || amount <= 0) {
  throw new functions.https.HttpsError('invalid-argument', 'Invalid bid data');
}

const auctionRef = admin.firestore().collection('auctions').doc(auctionId);
const userRef = admin.firestore().collection('users').doc(userId);

const result = await admin.firestore().runTransaction(async (transaction) => {
  const [auctionSnap, userSnap] = await Promise.all([
    transaction.get(auctionRef),
    transaction.get(userRef),
  ]);

  if (!auctionSnap.exists) {
    throw new functions.https.HttpsError('not-found', 'Auction not found');
  }

  const auction = auctionSnap.data();
  const userData = userSnap.data() || {};

  // BUG FIX: role is now checked server-side. The original client code
  // inferred "supplier" simply as "not the buyer", so a user with a
  // `buyer` role on their OWN profile could still bid on someone else's
  // auction, which is not a valid marketplace role.
  if (userData.role !== 'supplier') {
    throw new functions.https.HttpsError('permission-denied', 'Only suppliers can bid');
  }

  if (auction.status !== 'active') {
    throw new functions.https.HttpsError('failed-precondition', 'Auction is not active');
  }

  if (new Date(auction.deadline).getTime() < Date.now()) {
    throw new functions.https.HttpsError('failed-precondition', 'Auction deadline has passed');
  }

  if (auction.buyerId === userId) {
    throw new functions.https.HttpsError('failed-precondition', 'Buyer cannot bid on their own auction');
  }

  const currentPaise = toPaise(auction.currentPrice);
  const decrementPaise = toPaise(auction.minimumBidDecrement);
  const amountPaise = toPaise(amount);
  const maxBidPaise = currentPaise - decrementPaise;

  if (amountPaise >= currentPaise) {
    throw new functions.https.HttpsError(
      'failed-precondition',
      `Bid must be less than current price (₹${auction.currentPrice.toFixed(2)})`
    );
  }
});

```

```

    }

    if (amountPaise > maxBidPaise) {
      throw new functions.https.HttpsError(
        'failed-precondition',
        `Maximum allowed bid is ₹${(maxBidPaise / 100).toFixed(2)}`
      );
    }

    // BLIND BIDDING FIX: assign a stable alias per supplier instead of
    // exposing supplierName in the public bid history.
    const aliasMap = auction.supplierAliases || {};
    let alias = aliasMap[userId];
    if (!alias) {
      const nextIndex = Object.keys(aliasMap).length;
      alias = `Supplier ${String.fromCharCode(65 + (nextIndex % 26))}`; // A, B, C...
      aliasMap[userId] = alias;
    }

    const bidData = {
      supplierId: userId,
      supplierAlias: alias,
      amount: amountPaise / 100,
      timestamp: new Date().toISOString(),
    };

    const updates = {
      currentPrice: amountPaise / 100,
      bidCount: admin.firestore.FieldValue.increment(1),
      bids: admin.firestore.FieldValue.arrayUnion(bidData),
      supplierAliases: aliasMap,
    };

    // ANTI-SNIPING FIX applied here
    const deadlineMs = new Date(auction.deadline).getTime();
    const extensionsUsed = auction.extensionsUsed || 0;
    if (deadlineMs - Date.now() < SNIPE_WINDOW_MS && extensionsUsed < MAX_EXTENSIONS) {
      updates.deadline = new Date(deadlineMs + SNIPE_EXTENSION_MS).toISOString();
      updates.extensionsUsed = extensionsUsed + 1;
    }

    transaction.update(auctionRef, updates);

    return { bidData, extended: !!updates.deadline };
  });

  return {
    success: true,
    message: result.extended
      ? 'Bid placed – deadline extended due to last-minute activity'
      : 'Bid placed successfully',
    data: result.bidData,
  };
});

```

5.3 functions/src/closeAuction.js

```
const functions = require('firebase-functions');
const admin = require('firebase-admin');

exports.closeExpiredAuctions = functions.pubsub
  .schedule('every 5 minutes')
  .onRun(async () => {
    const now = new Date();

    const auctionsSnapshot = await admin.firestore()
      .collection('auctions')
      .where('status', '==', 'active')
      .get();

    // BUG FIX: a single batch has a 500-write limit. The original code put
    // every closing auction into one batch with no chunking, which would
    // silently throw and abort ALL closures once more than ~500 auctions
    // were due at once. We now chunk into batches of 400 to stay well
    // under the limit (each closing auction can use up to 2 writes: the
    // status update and, if it has bids, the winner subcollection doc).
    const dueAuctions = [];
    auctionsSnapshot.forEach((doc) => {
      const auction = doc.data();
      if (new Date(auction.deadline) < now) {
        dueAuctions.push({ id: doc.id, ...auction });
      }
    });

    const CHUNK_SIZE = 400;
    for (let i = 0; i < dueAuctions.length; i += CHUNK_SIZE) {
      const chunk = dueAuctions.slice(i, i + CHUNK_SIZE);
      const batch = admin.firestore().batch();

      for (const auction of chunk) {
        const ref = admin.firestore().collection('auctions').doc(auction.id);
        batch.update(ref, {
          status: 'closed',
          closedAt: now.toISOString(),
        });

        // BUG FIX: the original function never determined a winner. In this
        // "lowest qualified bid wins" model, currentPrice already holds the
        // lowest accepted bid, so the last entry in `bids` is the winner -
        // but that was never surfaced anywhere for the buyer to act on.
        if (auction.bids && auction.bids.length > 0) {
          const winningBid = auction.bids[auction.bids.length - 1];
          const winnerRef = ref.collection('private').doc('winner');
          batch.set(winnerRef, {
            winnerId: winningBid.supplierId,
            winnerAlias: winningBid.supplierAlias,
            winningAmount: winningBid.amount,
            decidedAt: now.toISOString(),
          });
        }
      }
    }
  });
```

```

    }

    await batch.commit();
  }

  console.log(`Closed ${dueAuctions.length} expired auctions`);
  return null;
});

```

5.4 functions/src/notifications.js (previously missing)

```

const functions = require('firebase-functions');
const admin = require('firebase-admin');

// BUG FIX: functions/src/index.js exports `sendBidNotification` and
// `sendAuctionClosedNotification` from this file, but the file was missing
// from the original handoff - `require('./notifications')` would throw
// MODULE_NOT_FOUND and the ENTIRE functions deployment (including
// validateBid and closeExpiredAuctions) would fail to boot, since one bad
// export crashes the whole functions index in Node.

exports.sendBidNotification = functions.firestore
  .document('auctions/{auctionId}')
  .onUpdate(async (change, context) => {
    const before = change.before.data();
    const after = change.after.data();

    if ((after.bidCount || 0) <= (before.bidCount || 0)) {
      return null; // not a new bid
    }

    const buyerDoc = await admin.firestore().collection('users').doc(after.buyerId).get();
    const buyer = buyerDoc.data();
    if (!buyer || !buyer.fcmToken) return null;

    return admin.messaging().send({
      token: buyer.fcmToken,
      notification: {
        title: 'New bid received',
        body: `${after.title}: new price ₹${Number(after.currentPrice).toFixed(2)}`,
      },
      data: { auctionId: context.params.auctionId, type: 'new_bid' },
    }).catch((err) => {
      console.error('Failed to send bid notification:', err);
      return null;
    });
  });

exports.sendAuctionClosedNotification = functions.firestore
  .document('auctions/{auctionId}')
  .onUpdate(async (change, context) => {
    const before = change.before.data();
    const after = change.after.data();

```

```

if (before.status === after.status || after.status !== 'closed') {
  return null;
}

const recipients = [after.buyerId];
const winnerDoc = await admin.firestore()
  .collection('auctions').doc(context.params.auctionId)
  .collection('private').doc('winner').get();

if (winnerDoc.exists) {
  recipients.push(winnerDoc.data().winnerId);
}

const userDocs = await Promise.all(
  recipients.map((uid) => admin.firestore().collection('users').doc(uid).get())
);

const sends = userDocs
  .filter((doc) => doc.exists && doc.data().fcmToken)
  .map((doc) => admin.messaging().send({
    token: doc.data().fcmToken,
    notification: {
      title: 'Auction closed',
      body: `${after.title} has closed at ₹${Number(after.currentPrice).toFixed(2)}`,
    },
    data: { auctionId: context.params.auctionId, type: 'auction_closed' },
  })).catch((err) => {
    console.error('Failed to send close notification:', err);
    return null;
  }));

await Promise.all(sends);
return null;
});

```

5.5 client/src/components/Auction/BidForm.jsx

```

import { useState } from 'react';
import { getFunctions, httpsCallable } from 'firebase/functions';
import app from '../../firebase/config';

// BUG FIX: the original BidForm ran `runTransaction` directly against
// Firestore from the browser. That duplicated (and could drift from) the
// server-side rules in validateBid.js, and – combined with the old,
// over-permissive Firestore rules – meant a modified client could write an
// illegal bid straight to the database. All bid submission now goes through
// the validateBid Cloud Function, which is the single source of truth.
const functions = getFunctions(app);
const validateBidFn = httpsCallable(functions, 'validateBid');

const BidForm = ({ auctionId, currentPrice, minDecrement, onBidStart, onBidEnd }) => {
  const [bidAmount, setBidAmount] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

```

```

const [notice, setNotice] = useState('');

const maxBid = currentPrice - minDecrement;

const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setNotice('');

  const amount = parseFloat(bidAmount);

  // BUG FIX: parseFloat('') is NaN, and NaN comparisons are always false,
  // so the old code let an empty/garbage bid amount slip past the
  // `amount >= currentPrice` guard and reach the transaction.
  if (!bidAmount || Number.isNaN(amount) || amount <= 0) {
    setError('Enter a valid bid amount');
    return;
  }

  if (amount >= currentPrice) {
    setError(`Bid must be less than current price (₹${currentPrice.toFixed(2)})`);
    return;
  }

  if (amount > maxBid) {
    setError(`Maximum allowed bid is ₹${maxBid.toFixed(2)} (current price - min decrement)`);
    return;
  }

  setLoading(true);
  onBidStart();

  try {
    const result = await validateBidFn({ auctionId, amount });
    setBidAmount('');
    setNotice(result.data.message);
  } catch (err) {
    // Cloud Function HttpsError messages arrive in err.message
    setError(err.message || 'Could not place bid');
  } finally {
    setLoading(false);
    onBidEnd();
  }
};

return (
  <div className="bg-white rounded-lg shadow p-6 mb-6">
    <h3 className="text-xl font-semibold mb-4">Place Your Bid</h3>
    <form onSubmit={handleSubmit}>
      <div className="flex flex-col md:flex-row gap-4 items-end">
        <div className="flex-1">
          <label className="block text-sm font-medium mb-1">
            Your Bid (Max: ₹${maxBid.toFixed(2)})
          </label>
          <input

```



```

        type="number"
        value={bidAmount}
        onChange={(e) => setBidAmount(e.target.value)}
        step="0.01"
        min="0.01"
        max={maxBid}
        className="w-full p-2 border rounded"
        placeholder="Enter bid amount"
        required
      />
    </div>
    <button
      type="submit"
      disabled={loading}
      className="bg-green-600 text-white px-6 py-2 rounded hover:bg-green-700 disabled:opacity-50"
      >
      {loading ? 'Placing Bid...' : 'Submit Bid'}
    </button>
  </div>
  {error && <div className="mt-3 text-red-600 text-sm">{error}</div>}
  {notice && <div className="mt-3 text-green-700 text-sm">{notice}</div>}
  <div className="mt-2 text-sm text-gray-500">
    Current price: ₹{currentPrice.toFixed(2)} • Minimum decrement: ₹{minDecrement.toFixed(2)}
  </div>
</form>
</div>
);
};

export default BidForm;

```

5.6 client/src/components/Auction/AuctionDetail.jsx

```

import { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { db } from '../../firebase/config';
import { doc, onSnapshot } from 'firebase/firestore';
import { useAuth } from '../../context/AuthContext';
import BidForm from './BidForm';
import BidHistory from './BidHistory';

const AuctionDetail = () => {
  const { id } = useParams();
  const { user } = useAuth();
  const [auction, setAuction] = useState(null);
  const [role, setRole] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [isBidding, setIsBidding] = useState(false);

  useEffect(() => {
    const unsubscribe = onSnapshot(doc(db, 'auctions', id), (snap) => {
      if (snap.exists()) {
        setAuction({ id: snap.id, ...snap.data() });
      }
    });
  }, [id]);

```

```

        setLoading(false);
    } else {
        setError('Auction not found');
        setLoading(false);
    }
}, (err) => {
    setError(err.message);
    setLoading(false);
});

return () => unsubscribe();
}, [id]);

// BUG FIX: role used to be inferred client-side as "not the buyer of THIS
// auction" – which meant a user whose own profile role is "buyer" could
// still see and use the bid form on someone else's auction. We now read
// the user's actual role from their profile document. (This is a
// convenience/UX check only; the real enforcement is server-side in
// validateBid.js, which the Firestore rules now require.)
useEffect(() => {
    if (!user) {
        setRole(null);
        return;
    }
    const unsubscribe = onSnapshot(doc(db, 'users', user.uid), (snap) => {
        setRole(snap.exists() ? snap.data().role : null);
    });
    return () => unsubscribe();
}, [user]);

const getTimeRemaining = (deadline) => {
    const diff = new Date(deadline) - new Date();
    if (diff <= 0) return 'Closed';
    const hours = Math.floor(diff / (1000 * 60 * 60));
    const minutes = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60));
    return `${hours}h ${minutes}m`;
};

const isBuyer = auction?.buyerId === user?.uid;
const isSupplier = role === 'supplier' && !isBuyer;
const isActive = auction?.status === 'active';

if (loading) return <div className="text-center py-10">Loading auction details...</div>;
if (error) return <div className="text-center py-10 text-red-600">{error}</div>;
if (!auction) return <div className="text-center py-10">Auction not found</div>;

return (
    <div className="max-w-4xl mx-auto">
        <div className="bg-white rounded-lg shadow p-6 mb-6">
            <div className="flex justify-between items-start mb-4">
                <h1 className="text-3xl font-bold">{auction.title}</h1>
                <span className={`px-3 py-1 rounded ${
                    auction.status === 'active' ? 'bg-green-100 text-green-800' : 'bg-gray-100 text-
                }`}>
                    {auction.status}
                </span>
            </div>
        </div>
    </div>

```

```

    </span>
  </div>

  <div className="grid grid-cols-2 gap-4 mb-4">
    <div>
      <p className="text-sm text-gray-500">Category</p>
      <p>{auction.category}</p>
    </div>
    <div>
      <p className="text-sm text-gray-500">Current Price</p>
      <p className="text-2xl font-bold text-blue-600">₹{auction.currentPrice?.toFixed(
    </div>
    <div>
      <p className="text-sm text-gray-500">Starting Price</p>
      <p>₹{auction.startingPrice?.toFixed(2)}</p>
    </div>
    <div>
      <p className="text-sm text-gray-500">Time Remaining</p>
      <p className="text-orange-600 font-semibold">{getTimeRemaining(auction.deadline)}
    </div>
    <div>
      <p className="text-sm text-gray-500">Quantity</p>
      <p>{auction.quantity} {auction.unit}</p>
    </div>
    <div>
      <p className="text-sm text-gray-500">Total Bids</p>
      <p>{auction.bidCount || 0}</p>
    </div>
  </div>

  <div className="mb-4">
    <p className="text-sm text-gray-500">Description</p>
    <p className="mt-1">{auction.description}</p>
  </div>

  {auction.specifications && (
    <div className="mb-4 p-4 bg-gray-50 rounded">
      <p className="text-sm text-gray-500">Specifications</p>
      <p className="mt-1 whitespace-pre-wrap">{auction.specifications}</p>
    </div>
  )}

  {isBuyer && isActive && (
    <div className="bg-yellow-50 p-4 rounded border border-yellow-200">
      <p className="text-sm">You are the buyer for this auction. Suppliers are bidding
    </div>
  )}
</div>

{isSupplier && isActive && (
  <BidForm
    auctionId={id}
    currentPrice={auction.currentPrice}
    minDecrement={auction.minimumBidDecrement}
    onBidStart={() => setIsBidding(true)}

```

```

        onBidEnd={() => setIsBidding(false)}
      />
    )}

    <BidHistory auctionId={id} />
  </div>
);
};

export default AuctionDetail;

```

5.7 client/src/components/Auction/BidHistory.jsx

```

import { useState, useEffect } from 'react';
import { db } from '../../firebase/config';
import { doc, onSnapshot } from 'firebase/firestore';

const BidHistory = ({ auctionId }) => {
  const [bids, setBids] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onSnapshot(doc(db, 'auctions', auctionId), (snap) => {
      if (snap.exists()) {
        setBids(snap.data().bids || []);
        setLoading(false);
      }
    }, (err) => {
      console.error('Error fetching bids:', err);
      setLoading(false);
    });

    return () => unsubscribe();
  }, [auctionId]);

  if (loading) return <div className="text-center py-4">Loading bid history...</div>;

  if (bids.length === 0) {
    return (
      <div className="bg-white rounded-lg shadow p-6">
        <h3 className="text-xl font-semibold mb-4">Bid History</h3>
        <p className="text-gray-500">No bids yet. Be the first to bid!</p>
      </div>
    );
  }

  // BUG FIX: the original code just reversed the array to show "latest
  // first", which only works if bids always land in submission order.
  // `arrayUnion` writes are ordered by the server, but relying on that is
  // fragile – sort explicitly by timestamp instead.
  const sortedBids = [...bids].sort(
    (a, b) => new Date(b.timestamp) - new Date(a.timestamp)
  );

```

```

return (
  <div className="bg-white rounded-lg shadow p-6">
    <h3 className="text-xl font-semibold mb-4">Bid History ({bids.length} bids)</h3>
    <div className="overflow-x-auto">
      <table className="w-full">
        <thead className="bg-gray-50">
          <tr>
            <th className="px-4 py-2 text-left text-sm font-medium text-gray-500">#</th>
            {/* BUG FIX: was supplierName – leaked competing suppliers' real identities to each other, breaking blind-bidding. */}
            <th className="px-4 py-2 text-left text-sm font-medium text-gray-500">Supplier
            <th className="px-4 py-2 text-right text-sm font-medium text-gray-500">Amount
            <th className="px-4 py-2 text-right text-sm font-medium text-gray-500">Time</th>
          </tr>
        </thead>
        <tbody>
          {sortedBids.map((bid, index) => (
            <tr key={`-${bid.supplierId}-${bid.timestamp}`} className="border-t">
              <td className="px-4 py-2 text-sm">{bids.length - index}</td>
              <td className="px-4 py-2 text-sm">{bid.supplierAlias || 'Supplier'}</td>
              <td className="px-4 py-2 text-sm text-right font-medium">{bid.amount.toFixed(2)}</td>
              <td className="px-4 py-2 text-sm text-right text-gray-500">
                {new Date(bid.timestamp).toLocaleTimeString()}
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
);
};

export default BidHistory;

```

5.8 test_bid_logic.js (regression harness used in §3)

```

// Regression test harness for the fixed bid-validation logic (mirrors
// functions/src/validateBid.js). Run with: node test_bid_logic.js

const toPaise = (rupees) => Math.round(rupees * 100);
const SNIPE_WINDOW_MS = 2 * 60 * 1000;
const SNIPE_EXTENSION_MS = 2 * 60 * 1000;
const MAX_EXTENSIONS = 10;

function placeBid(auction, userId, userRole, amount, now = Date.now()) {
  if (typeof amount !== 'number' || !isFinite(amount) || amount <= 0) {
    throw new Error('invalid-argument: Invalid bid data');
  }
  if (userRole !== 'supplier') {
    throw new Error('permission-denied: Only suppliers can bid');
  }
  if (auction.status !== 'active') {
    throw new Error('failed-precondition: Auction is not active');
  }
}

```

```

}
if (new Date(auction.deadline).getTime() < now) {
  throw new Error('failed-precondition: Auction deadline has passed');
}
if (auction.buyerId === userId) {
  throw new Error('failed-precondition: Buyer cannot bid on their own auction');
}

const currentPaise = toPaise(auction.currentPrice);
const decrementPaise = toPaise(auction.minimumBidDecrement);
const amountPaise = toPaise(amount);
const maxBidPaise = currentPaise - decrementPaise;

if (amountPaise >= currentPaise) {
  throw new Error(`failed-precondition: Bid must be less than current price (${auction.cur
}
if (amountPaise > maxBidPaise) {
  throw new Error(`failed-precondition: Maximum allowed bid is ${maxBidPaise / 100}.toFix
}

const aliasMap = auction.supplierAliases || {};
let alias = aliasMap[userId];
if (!alias) {
  alias = `Supplier ${String.fromCharCode(65 + Object.keys(aliasMap).length)}`;
  aliasMap[userId] = alias;
}

auction.currentPrice = amountPaise / 100;
auction.bidCount = (auction.bidCount || 0) + 1;
auction.bids = [...(auction.bids || []), { supplierId: userId, supplierAlias: alias, amount
auction.supplierAliases = aliasMap;

const deadlineMs = new Date(auction.deadline).getTime();
const extensionsUsed = auction.extensionsUsed || 0;
let extended = false;
if (deadlineMs - now < SNIPE_WINDOW_MS && extensionsUsed < MAX_EXTENSIONS) {
  auction.deadline = new Date(deadlineMs + SNIPE_EXTENSION_MS).toISOString();
  auction.extensionsUsed = extensionsUsed + 1;
  extended = true;
}

return { auction, extended };
}

function test(name, fn) {
  try {
    fn();
    console.log(`PASS ${name}`);
  } catch (e) {
    console.log(`FAIL ${name} -> ${e.message}`);
    process.exitCode = 1;
  }
}

function expectThrow(fn, substr) {
  try {

```

```

    fn();
  } catch (e) {
    if (!e.message.includes(substr)) throw new Error(`wrong error: ${e.message}`);
    return;
  }
  throw new Error('expected throw, none occurred');
}

const baseAuction = () => ({
  buyerId: 'buyer1',
  status: 'active',
  currentPrice: 1000,
  minimumBidDecrement: 10,
  startingPrice: 1000,
  deadline: new Date(Date.now() + 60 * 60 * 1000).toISOString(), // 1hr out
  bids: [],
  bidCount: 0,
});

test('valid bid lowers price and records alias', () => {
  const a = baseAuction();
  const { auction } = placeBid(a, 'sup1', 'supplier', 985);
  if (auction.currentPrice !== 985) throw new Error('price not updated');
  if (auction.bids[0].supplierAlias !== 'Supplier A') throw new Error('alias wrong');
});

test('second supplier gets alias B', () => {
  const a = baseAuction();
  placeBid(a, 'sup1', 'supplier', 985);
  const { auction } = placeBid(a, 'sup2', 'supplier', 970);
  if (auction.bids[1].supplierAlias !== 'Supplier B') throw new Error('alias wrong');
});

test('bid equal to current price is rejected', () => {
  const a = baseAuction();
  expectThrow(() => placeBid(a, 'sup1', 'supplier', 1000), 'less than current price');
});

test('bid violating min decrement is rejected', () => {
  const a = baseAuction(); // max allowed = 990
  expectThrow(() => placeBid(a, 'sup1', 'supplier', 995), 'Maximum allowed bid');
});

test('bid exactly at max decrement boundary is accepted (float-precision case)', () => {
  const a = baseAuction();
  a.currentPrice = 100.10;
  a.minimumBidDecrement = 0.10;
  // 100.10 - 0.10 = 100.0 in exact decimal, but 100.10 - 0.10 === 100.00000000000001 in JS
  const { auction } = placeBid(a, 'sup1', 'supplier', 100.00);
  if (auction.currentPrice !== 100.00) throw new Error('boundary bid incorrectly rejected/alias');
});

test('buyer cannot bid on own auction', () => {
  const a = baseAuction();
  expectThrow(() => placeBid(a, 'buyer1', 'supplier', 900), 'own auction');
});

```

```

});

test('buyer-role user cannot bid even on someone else\'s auction', () => {
  const a = baseAuction();
  expectThrow(() => placeBid(a, 'sup1', 'buyer', 900), 'Only suppliers can bid');
});

test('bid after deadline is rejected', () => {
  const a = baseAuction();
  a.deadline = new Date(Date.now() - 1000).toISOString();
  expectThrow(() => placeBid(a, 'sup1', 'supplier', 900), 'deadline has passed');
});

test('last-minute bid extends deadline (anti-sniping)', () => {
  const a = baseAuction();
  a.deadline = new Date(Date.now() + 60 * 1000).toISOString(); // 1 min out, inside 2-min wi
  const before = new Date(a.deadline).getTime();
  const { extended, auction } = placeBid(a, 'sup1', 'supplier', 900);
  if (!extended) throw new Error('should have extended');
  if (new Date(auction.deadline).getTime() <= before) throw new Error('deadline did not move
});

test('extension count is capped', () => {
  const a = baseAuction();
  a.extensionsUsed = 10;
  a.deadline = new Date(Date.now() + 60 * 1000).toISOString();
  const { extended } = placeBid(a, 'sup1', 'supplier', 900);
  if (extended) throw new Error('should not extend past cap');
});

test('NaN / empty amount rejected', () => {
  const a = baseAuction();
  expectThrow(() => placeBid(a, 'sup1', 'supplier', NaN), 'invalid-argument');
});

```


6. ABO Stress Platform — Wireframes (unchanged, reference only)

No code accompanied this section in the original hand-off — it is five ASCII wireframe screens (Executive Dashboard, Team Analytics, Digital Twin Simulation, Individual Employee View, Biological Aging Monitor). Since there's no code here, there's nothing to bug-fix; the wireframes are preserved below for continuity with the original document. The original transcript was cut off mid-way through Screen 5, which is reproduced as far as it was provided.

(Wireframes omitted from this report for brevity — available on request, or refer to the original working document. If you'd like, I can turn these into an actual Figma-style interactive prototype or a set of properly rendered mockup images in a follow-up.)